RoboCupJunior Rescue Maze 2025

Team Description Paper

Таёжные Ёжики

Работу выполнили:

ученица МБОУ лицей при ТПУ г. Томска — М.С. Цыганкова

ученик ОГБОУ «ТФТЛ» — Г.А. Пильщиков

Наставник проекта:

Заместитель директора по информационным технологиям ОГБОУ «ТФТЛ» — С.В. Косаченко

Томск 2025

АНОТАЦИЯ

Данная работа содержит описание работы над роботом для решения задач RCJ Rescue Maze. Особенностью нашего робота заключается в самодельные элементы конструкции и использовании микрокомпьютера Raspberry Pi 4 для определения жертв и микроконтроллера Raspberry Pi PICO. Стратегии выполнения самого задания мы выбрали следующую. Наш робот передвигается по лабиринту по приоритетам прямо, лево право. Во время движения он просчитывает свою текущую координату относительно клетки старта и заносит информацию о посещении, наличие/присутствии стен, жертв, болота/ямы в карту, при этом робот в может пренебрегать приоритетом для изучения ранее не изученных клеток. Эта стратегия позволяет изучить как можно больше территории и найти как можно больше жертв.



Рисунок 1 — фото команды

Опыт участия и успехи команды в различных робототехнических соревнованиях и фестивалях

- RoboScience Hackathon 2024 - 1 место в состязании Robocup Junior Rescue Maze
- Международный фестиваль Робофинист 2022 — 3 место в категории Robocup Junior Rescue Maze, г.Санкт-Петербург
- Кубок Губернатора Томской области в состязании роботов с техническим зрением памяти Виктора Ширшина (2024 - 1 место, 2023 - 1 место, 2022 - 2 место)
- Региональная олимпиада по образовательной робототехнике школьников Томской области — 2 место в состязании Robocup Junior Rescue Maze (2024 год), 1 место в состязании Robocup Junior Rescue Simulation Webots Erebus (2023 год)

- Открытый Российский чемпионат по робототехнике 2023 — 2 место в состязании Robocup Junior Rescue Simulation Webots Erebus
- Цыганкова Мария - дважды победитель во всероссийском конкурсе научно-технологических проектов “Большие вызовы” по направлении Освоение Арктики и мирового океана (2024 год), Беспилотные и логистические системы (2025 год)
- Цыганкова Мария - участник заключительного этапа ВСОШ по профилю Робототехника
- Пильщиков Григорий - призер во всероссийском конкурсе научно-технологических проектов “Большие вызовы” по направлении Освоение Арктики и мирового океана (2024 год),
- 2022 год — Участие Цыганковой Марии и Пильщикова Григория в фестивале подводной робототехнике «АкваРобоФест» в г.Асино

ВВЕДЕНИЕ

Предполагаемый результат проекта

При разработке проекта планировалось создать уменьшенный прототип аппарата, для полностью автономного проведения поисково-спасательной операции, анализируя и приспосабливаясь к окружающей среде, следовательно, в соответствии ГОСТ Р60, робота-спасателя для выполнения поставленных задач.

Команда для реализации проекта

1) Пильщиков Григорий - капитан команд, программист, конструктор, отвечает за разработку и распайку электрической части робота, подбор электрических компонентов, за разработку компьютерного зрения
2) Цыганкова Мария - член команды, программист, конструктор, отвечает за разработку корпуса робота, обеспечением компонентов сборки, деталей робота, осуществляет сборку робота, разрабатывает движения и ориентирование в лабиринте
3) Косаченко Сергей Викторович - наставник команды.

Цель работы

Разработать уменьшенный прототип автономного робота для помощи проведения поисково-спасательных операций.

Задачи

1. Изучить уже существующие решения, аналоги с похожим функционалом
2. Разработать работоспособный макет робота
3. Протестировать робота, проанализировать полученные результаты
4. Доработать минусы, появившиеся в результате тестирования

ОСНОВНАЯ ЧАСТЬ РАБОТЫ

План работ

1) Формулировка технического задания, расчет и моделирование конструкции.

Необходимые ресурсы/компоненты: ноутбук, программы для черчения.

Ответственный: Мария

Сроки: 01.2025 - 02.2025

Промежуточный результат: разработанная и рассчитанная конструкция робота, список необходимых для сборки компонентов

2) Расчет и моделирование печатной платы.

Необходимые ресурсы/компоненты: ноутбук, программы для черчения.

Ответственный: Григорий

Сроки: 01.2025 - 02.2025

Промежуточный результат: разработанная и рассчитанная электрическая часть робота, список необходимых для сборки компонентов

3) Сборка и тестирование шасси.

Необходимые ресурсы/компоненты: металлические несущие балки, двигатели с энкодорами, шестерни/колеса, гусеничная лента, набор инструментов.

Ответственный: Григорий

Сроки: 03.2025 - 04.2025

Промежуточный результат: готовое рабочее шасси с возможности регулировать натяжение гусениц без разбора механизма.

4) Сборка каркаса робота, установка контроллера.

Необходимые ресурсы/компоненты: контроллер, ЧПУ станок, фанера, гайки / шайбы / болтики, резьбовые шпильки, набор инструментов.

Ответственный: Мария

Сроки: 03.2025 - 04.2025

Промежуточный результат: готовый корпус робота.

5) Пайка и подключение электрических компонентов.

Необходимые ресурсы/компоненты: паяльник, канифоль, олово, флюс, третья рука, пинцет, соединительные провода, подобранные и приобретенные электрические компоненты.

Ответственный: Григорий

Сроки: 04.2025

Промежуточный результат: готовая ходовая часть робота.

6) Разработка ПО, тестирование начального прототипа.

Необходимые ресурсы/компоненты: ноутбук, кабель для программирования, место для

испытаний.

Ответственный: Мария

Сроки: 09.2024 - 12.2024

Промежуточный результат: начальный прототип робота с возможностью передвижения в замкнутых пространствах.

7) Установка камер, разработка системы компьютерного зрения.

Необходимые ресурсы/компоненты: ноутбук, микрокомпьютер, камеры, система подсветки камеры.

Ответственный: Григорий

Сроки: 04.2025 - 05.2025

Промежуточный результат: готовый прототип робота с системой обнаружения жертв.

8) Разработка систем навигации и картографирования.

Необходимые ресурсы/компоненты: ноутбук, кабель для программирования, симулятор для быстрого тестирования алгоритмов.

Ответственный: Мария

Сроки: 09.2024 - 12.2024, 05.2025

Промежуточный результат: экспериментальный прототип ПСР, готовый к тестированию.

9) Тестирование и анализ результатов.

Необходимые ресурсы/компоненты: ноутбук, кабель для программирования, полигон для испытаний, монитор для снятия телеметрии, монитор.

Ответственный: Мария, Григорий

Сроки: 05.2025

Промежуточный результат: выявленные плюсы/минусы текущей версии ПСР

10) Доработка и усовершенствование.

Необходимые ресурсы/компоненты: в зависимости от того, что будем улучшать.

Ответственный: Мария, Григорий

Сроки: 05.2025

ИТОГ: Экспериментальный прототип автономного работа для проведения поисково-спасательных операций.

Формулировка технического задания проекта, план интеграции проекта

Для простоты тестирования, за основу требований к роботу были взяты требования и регламенты различной сложности состязаний роботов-спасателей для студентов и школьников. Мы смоделировали спасательную миссию: робот должен по максимуму обследовать помещение (рисунок 2), построить его карту, отметить и сообщить об опасных или потенциально опасных местах, при обнаружении пострадавших, незамедлительно сообщить спасателям и скинуть комплект для оказания первой помощи, продолжить миссию, после окончании операции вернуться обратно к месту старта. Таким образом, нетрудно сформулировать и сами требования к роботу.

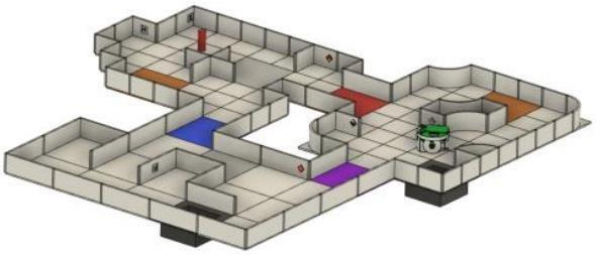


Рисунок 2 – модель поля проведения миссии

- Шасси робота должно быть достаточно прочным, надежным и проходимым, для преодоления препятствий различной трудности.
- Шины или гусеницы должны быть выполнены из нескользящего материала, для увеличения сцепления с поверхностью.
- Следует предусмотреть энкодеры на каждый из моторов.
- Робот должен обладать датчиками и сенсорами для перемещения по объекту и построении его карты.
- Робот должен контролировать обстановку не только вокруг себя, но и под самим роботом.
- Робот должен обладать камерами и системой компьютерного зрения для обнаружения подозрительных вещей.
- Для системы компьютерного зрения, необходимо выделить микрокомпьютер, задачей которого будет являться анализ изображения и отправка сигнала в случае обнаружения уже на сам контроллер.
- Питание контроллера и микрокомпьютера следует разделить, то есть необходимо предусмотреть раздельное питание с удобством быстрой смены аккумуляторов.
- Для устойчивости конструкции, необходимо стремиться сместить центр тяжести робота как можно ниже.
- Необходимо разработать статичные камеры для снижения риска сбоя кадра на тренировках.
- Для питания контроллера следует использовать высокотоковые аккумуляторы типа

18650.

- Необходимо предусмотреть стабилизатор питания для некоторых датчиков.
- Дополнительно, необходимо также предусмотреть систему выгрузки комплектов первой помощи.

Таким образом, была разработана следующая концепция робота. В качестве контроллера была выбрана плата Raspberry Pi PICO, так как она оказалась мощнее обычно Arduino. В качестве микрокомпьютера была выбрана Raspberry Pi 4, так как это полноценная замена компьютеру на Linux, которая позволяет выполнять все поставленные нами задачи. Конструкция робота предполагает несколько этажей с разным функционалом: на нулевом располагаются моторы, аккумуляторы на 12 В для питания движков, основная плата с практически всей электроникой, в том числе и RPI PICO, датчик цвета, который мы выбрали для контроля состояния поля под роботом; на первом — пауэрбанк на 5В для питания всех Raspberry Pi (далее - RPI), обратная сторона основной платы где располагаются клячи для включения/выключения робота и драйвера для моторов; на втором — RPI 4, 2 камеры для анализа жертв и лидар как замена инфракрасным дальномерам; на третьем — мотор для вращения лидара; на четвертом - выгрузка револьверного типа с возможностью выгружать комплекты в обе стороны. Выгрузка комплектов происходит за счет вращения револьверного колеса с комплектами. Шасси плоское, колесное, ход полноприводный. Весь корпус робота смоделирован и изготовлен из фанеры, а за счет спуска всего питания и электроники на первый и второй этаж центр тяжести робота оказался сильно смещен вниз, что улучшает его проходимость.

Описание стратегии

Робот перемещается в лабиринте по изначальному приоритету прямо, лево, право, но после захождения в тупик возвращается исследовать неисследованные пути, при этом выбор неизведанного прохода происходит по алгоритму право, лево, вперед. Такой алгоритм позволяет исследовать как можно большую часть лабиринта. При нахождении жертвы он привязывает ее к конкретной координате, что помогает избежать повторных обнаружений. Для самого же обнаружения у нас есть нейросеть, которая быстро реагирует в передвижении и алгоритм метод точек для более точного результата. Для экономии памяти бы дробим каждый байт на биты и оперируем ими. Таким образом, используя все возможности нашего робота, мы способны выполнить всю миссию.

Разработка механической части робота

Разработка шасси

Начальным этапом разработки робота стал этап разработки корпуса и внешнего вида робота. Сначала весь робот был спроектирован в программе FreeCAD (рисунок 3), продуманы все крепежи и составные части робота, после чего при помощи программы Inkscape (рисунок 6) были дополнительно разработаны и смоделированы векторные чертежи деталей конструкции, которые позже изготовили из 3-х миллиметровой фанеры на станке ЧПУ. Сам робот имеет этажный метод компоновки элементов и глобально имеет 4 платформы. Так первая платформа представляет из себя самостоятельно разработанную печатную плату, где располагается практически вся электроника, вторая снизу - крепеж для пауэрбанка, сверху - веб камеры и RPI 4 для компьютерного зрения, третья платформа - лидар, четвертая - механизм выгрузки.

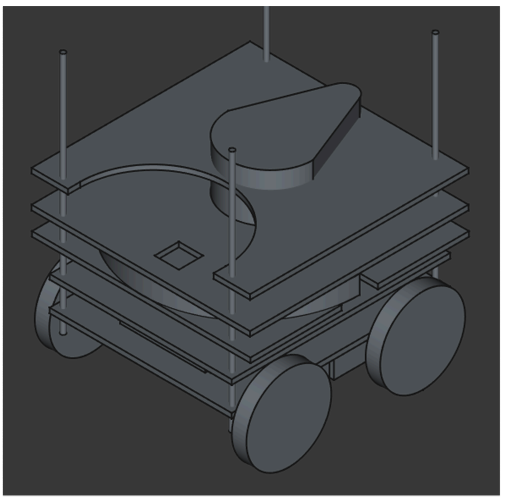


Рисунок 3 - 3D изначальная модель корпуса робота

Из интересных решений в конструкции стоит выделить корпус-крепеж для пауэрбанка (рисунок 5), так как он необходим для питания RPI 4 и имеет достаточно большие габариты и вес, а за счет этой детали конструкции мы сильно опускаем центр тяжести вниз. Также из интересных конструктивных решений у нас есть этаж для выгрузки комплектов (рисунок 4). Выгрузка револьверного типа позволяет с легкостью выгружать комплекты в обе стороны, занимая при этом минимум места.

Перед тестированием самого робота на каждом этапе его разработки проводились тестирования каждого элемента. Так, конструкция в первую очередь тестируется на прочность и если ее недостаточно, то элементы дорабатываем. Кроме того, при разработке шасси нам приходилось постоянно тестировать его проходимость - забираться на горки, проезжать через лежачие препятствия разной сложности, а также много экспериментировали с прохождением робота по различным поверхностям: гречка, пшено, лапша и другие крупы. В результате у нас есть робот с хорошей проходимостью.

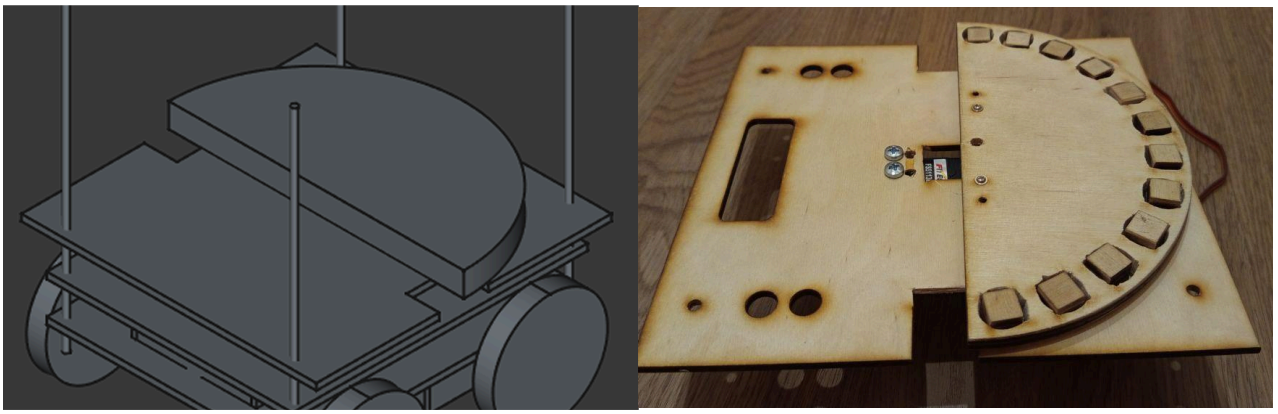


Рисунок 4 - механизм выгрузки комплектов

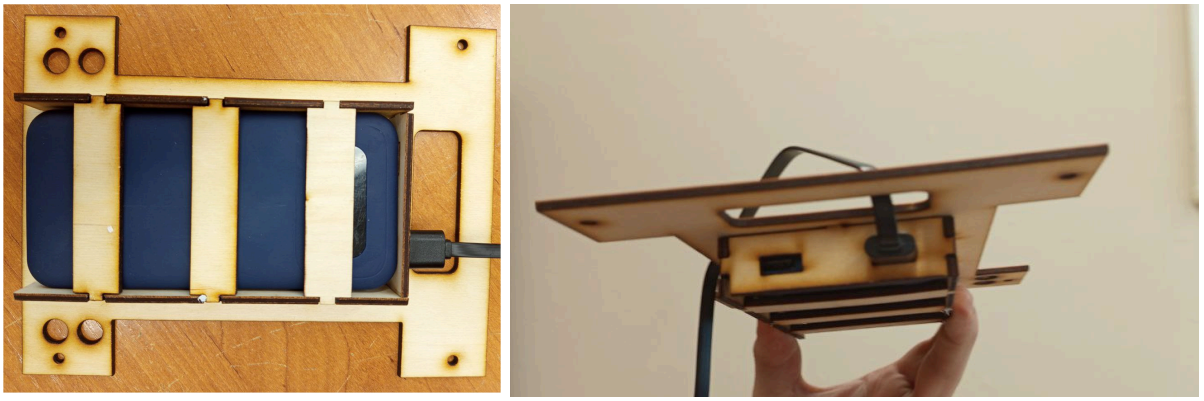


Рисунок 5 - крепеж пауэрбанка

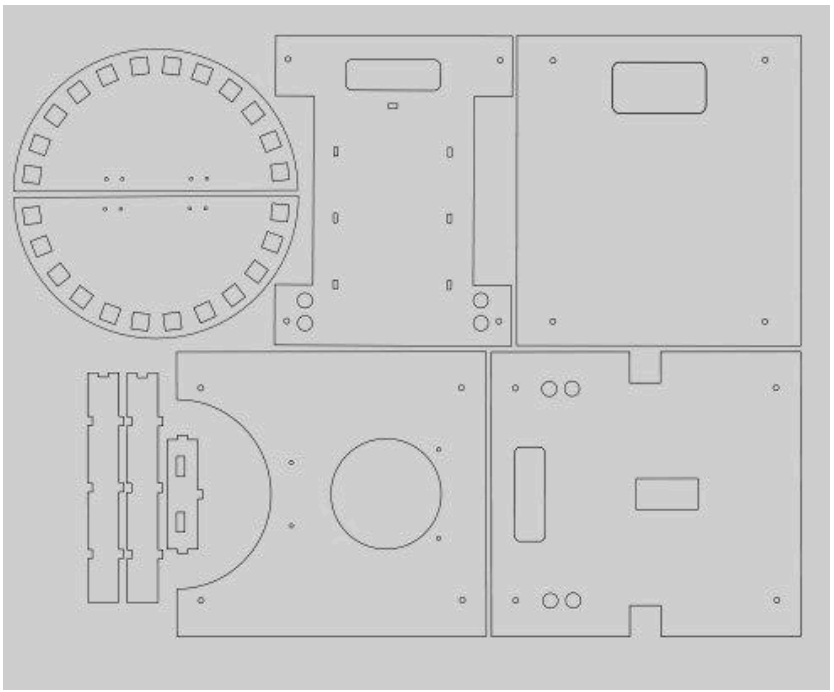


Рисунок 6 - SVG чертежи робота

Разработка электрической части робота

Пайка и подключение электрических компонентов

В качестве контроллера робота была выбрана плата Raspberry Pi PICO, так как она оказалась мощнее обычной Arduino и больше подходит для реализации картографирования, а также большее число аппаратных прерываний, что позволяет подключить к ней сразу 4 энкодера и сделать робота полноприводным. В качестве микрокомпьютера была выбрана Raspberry Pi 4. Также были выбрали инфракрасные дальномеры sharp 2y0a21 f 36, однако при развитии и модификации проекта их количества стало не хватать и они были заменены полноценным LDS лидаром Delta-2g для робота-пылесоса Xiaomi Mijia STYTJ02YM, 2 датчика цвета RGB TCS34725, веб камеры, считываемые RPI 4, которая анализирует данные с камеры и передает на RPI PICO, двигатели постоянного тока NoEnName_Null TT 9B, так как они достаточно легкие и мощные, к ним мы подключаем оптические энкодеры точностью до 360°. Драйвера для двигателей взяли стандартные L298N как наиболее доступные и распространенные. Данные между RPI мы передаем по протоколу UART. Для выгрузки используем сервопривод постоянного вращения Feetech FS5113R, потому что механизм изготовлен из фанеры и имеет достаточно большую силу трения между частями. Дополнительно из электроники у нас есть акселерометр и магнитометр, а также расширитель портов для RPI PICO.

После расчета напряжения для каждого электрического компонента робота, все электрические узлы были смоделированы в программе EasyEDA, после чего была заказана плата на Резоните, распаяна и собрана в общую схему, затем протестированы и закреплена как первый и нулевой этаж робота. Для питания 4 моторов были выбраны литий ионные аккумуляторы 18650, в количестве трех штук, так как таким образом для перемещения на вход моторов подается ≈12В, что является наиболее оптимальным. Для остальной печатной платы же выделили отдельное питание от обычного пауэрбанка на 5В.

Тестирование электроники же проходило в первую очередь на отсутствие коротких замыканий и корректность выдаваемых данных от датчиков, что также было экспериментально подтверждено. Для более удобного прохождения таких тестов было написано несколько программ для быстрого анализа показаний всех систем.

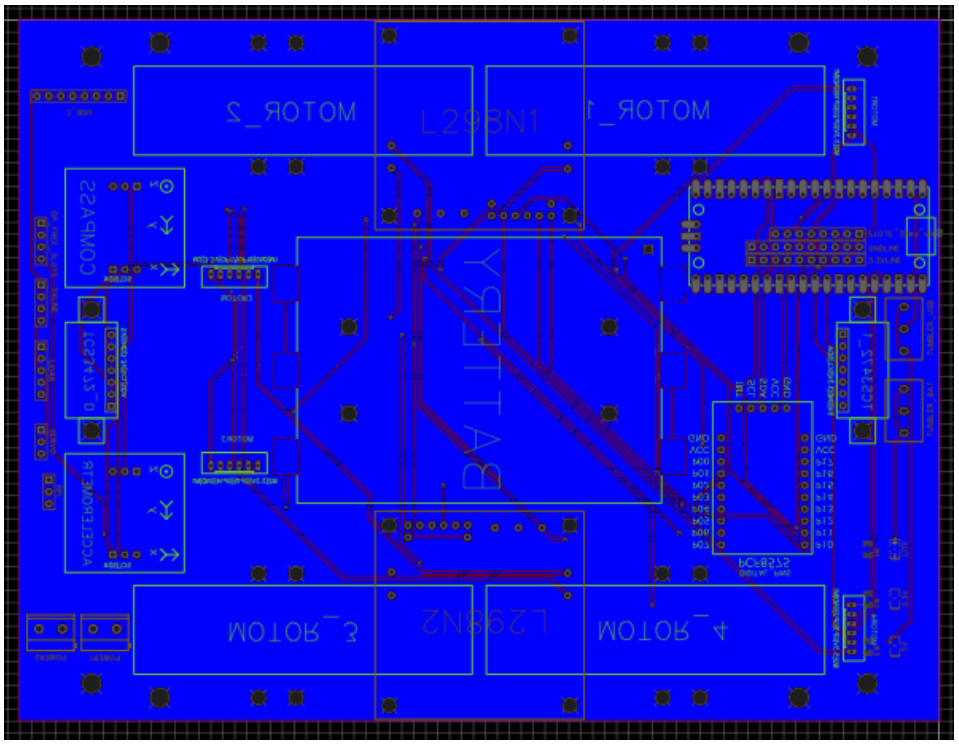


Рисунок 7 - нижний слой печатной платы

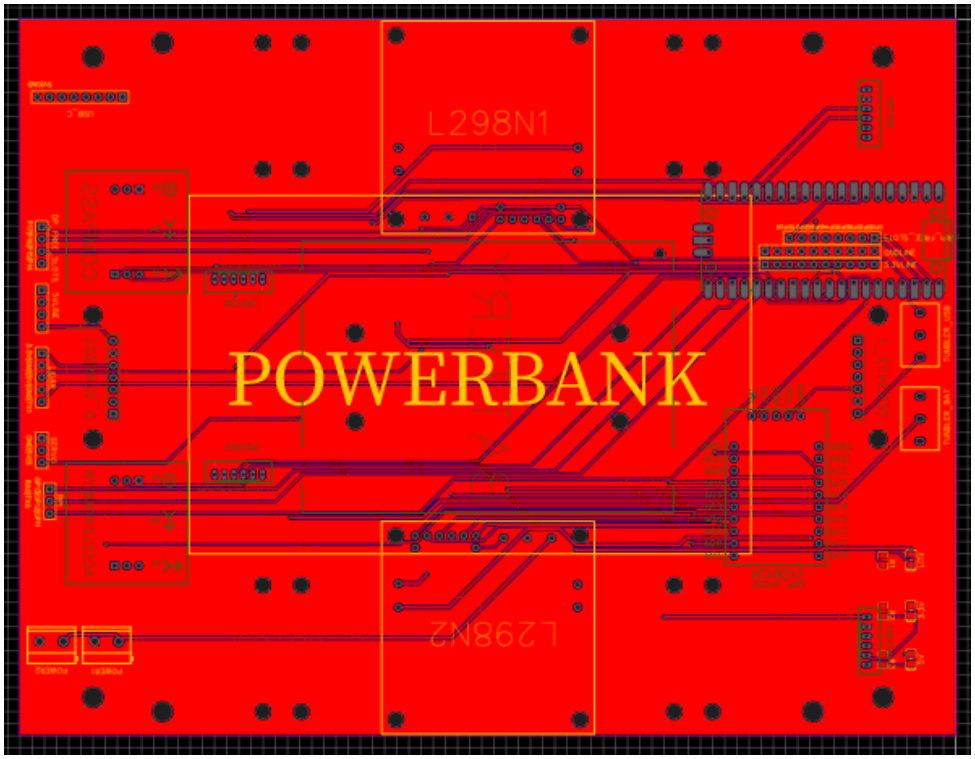


Рисунок 8 - верхний слой печатной платы

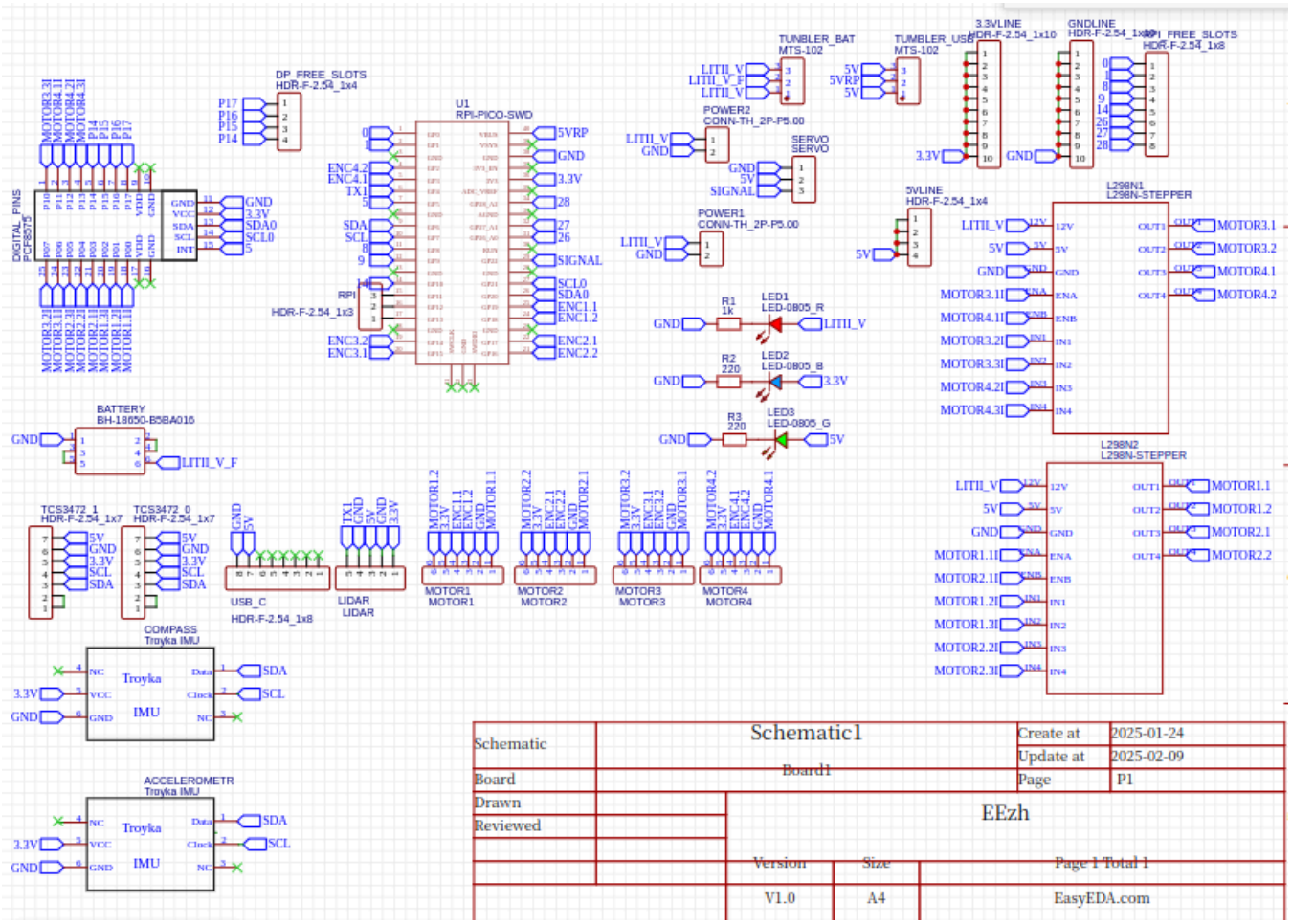


Рисунок 9 - проект печатной платы в EasyEDA

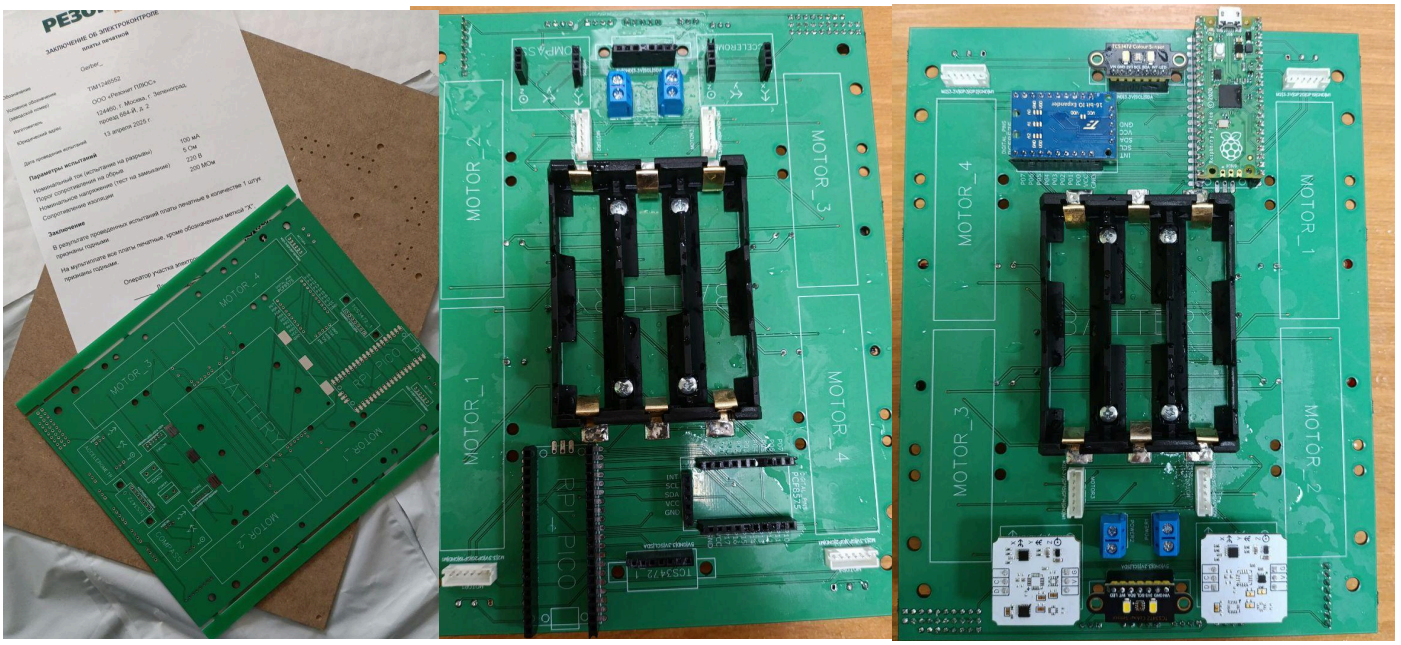


Рисунок 10 - процесс распайки печатной платы

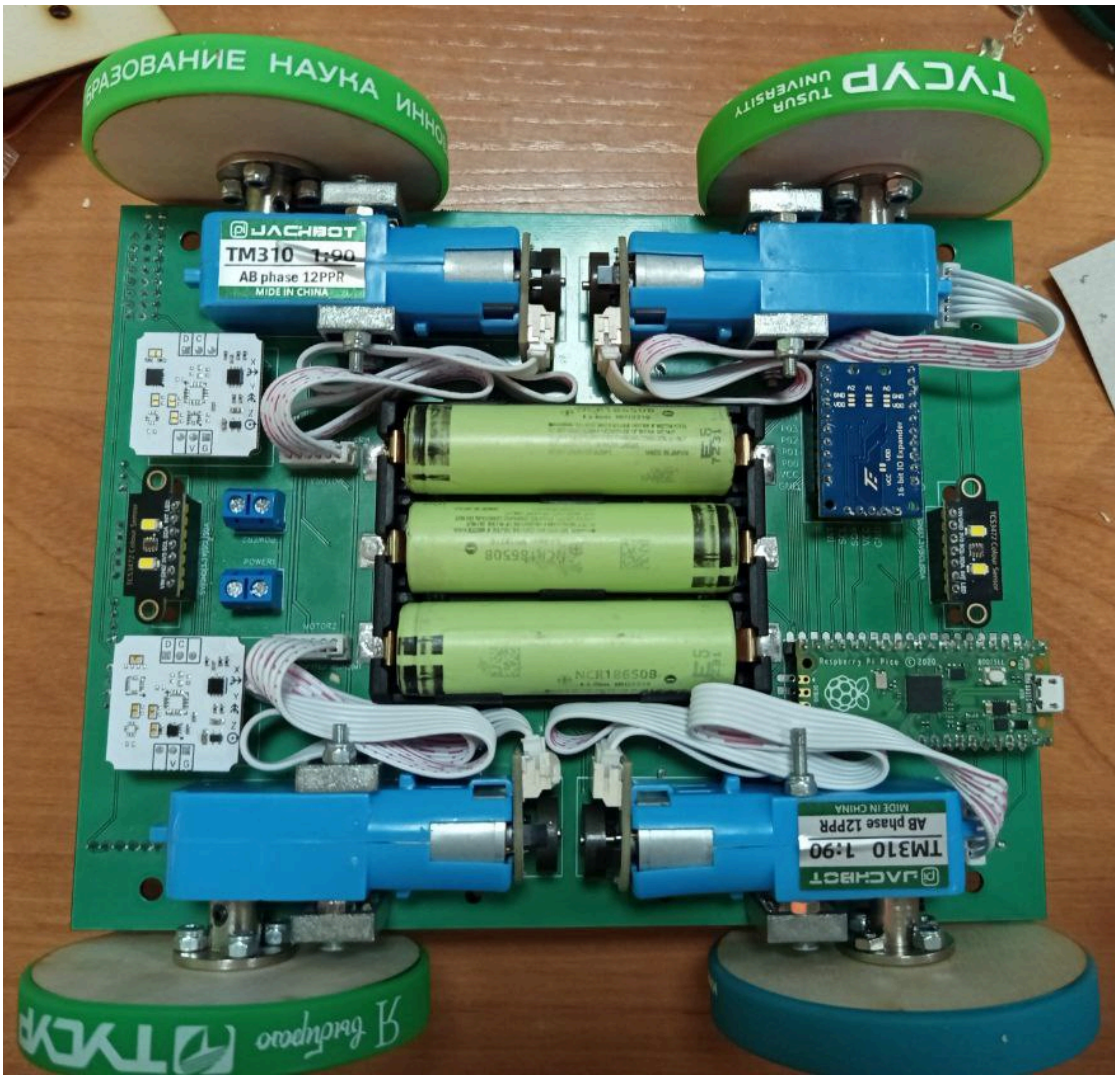


Рисунок 11 - итоговая печатная плата робота

Разработка программного обеспечения

Программное обеспечение робота делится на две глобальные задачи: алгоритмы передвижения и навигации (ПО для RPI PICO), а также алгоритмы компьютерного зрения (ПО для RPI 4).Контроллер прошивали через текстовую среду разработки Arduino IDE, так как она наиболее доступна и поддерживает библиотеки для RPI PICO. Этот алгоритм состоит из нескольких подпрограмм: передвижение, общение с RPI 4, построение карты и навигация по ней. Стоит отметить, что для простоты и точности тестирования ПО для контроллера робота, использовался стимулятор Webots Erebus, где были воссозданы некоторые полигон для проведения миссий и сам робот (рисунки 12, 13). Программу же писали в обычной текстовой среде программирования на языке C++, для удобства тестирования каждая подпрограмма

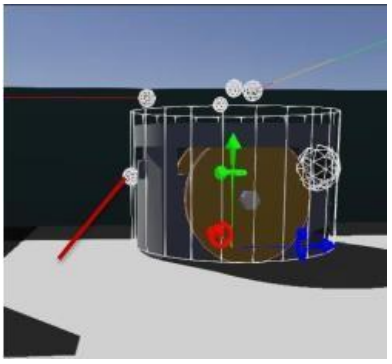


Рисунок 12 -модель робота в симуляторе

писалась как отдельная программа, и уже позже собиралась в одну основную. Контроль над передвижением осуществляется через оптические энкодеры, для простоты построения карты полигон как-бы мысленно делился на клетки 30х30 см. Во время движений, поворотов, выравниваний и так далее робот постоянно собирает и анализирует показание сторонних

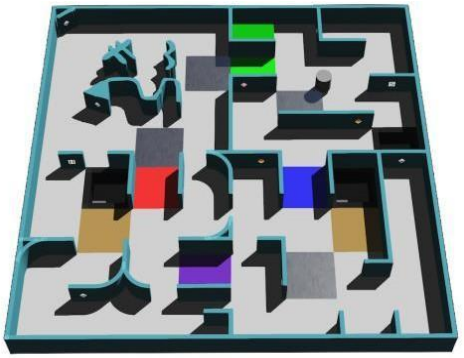


Рисунок 13 - модель лабиринта в симуляторе

датчиков (не лидара), что позволяет роботу оперативно реагировать на изменения в окружающей среде. Поворот же осуществляется при помощи акселерометра и компаса, и состоит из самого поворота и доворота (выравнивания) для точного попадания в найденный проход. Для более точного перемеще- ния в сильно замкнутых пространствах, используем пропорционально-дифференциальный регулятор, на основе показаний датчиков, что позволяет легко проходить подобные участки, не останавливаясь для выравнивания. Кроме того, при помощи акселерометра производится контроль над положением робота по оси OY, а именно, наличие ям или горок. Для обнаружения ям и болот мы также используем датчики цвета, которые способны определить яму еще до того, как центр тяжести перевесил в эту яму.

Картографирование выглядит следующим образом: карта представляет собой довольно большой массив из структур. Каждая структура состоит из нескольких однобайтных переменных, которые в свою очередь побитово делятся и в каждом бите лежит информация о наличие препятствий на интересующих нас градусах лидара, информацию о наличие жертв или иной информации для спасателей, а также количестве посещений этой ячейки. Для экономии памяти мы использовали по 2 байта на ячейку, так как для хранения информации о наличие препятствий, жертв, ям нам достаточно по одному биту памяти, следовательно при помощи побитового распределения памяти нам удалось сократить её использование в 40 раз, что помогло увеличить максимальный размер карты.

Фрагмент кода, для создания основных элементов хранения карты на C++:

```
struct myNode{
        uint8_t wall=0; //1 байт памяти отвечающий за объектах внутри клетки
        uint8_t count=0; // 1 байт памяти для количества посещений клетки
};
myNode mymap[40][40]; // сама карта

int cord_x=15, cord_y=25; // контроль текучей координаты
int napravlenie=2; // контроль направления робота относительно старта

uint8_t tam_l, tam_r;// для расчета смежных направлений относительно ведущего
```

Таким образом, во время посещения роботом следующей клетки, робот обновляет информацию о ячейке и пересчитывает побочные переменные, необходимые для

навигации. Сама же навигация робота строится на основе алгоритмов построения маршрутов в графе: DFS и алгоритм Дейкстры (блок схему можно прочитать в Приложении 1). Робот при посещении новой клетки смотрит, может ли он продолжать свое движение прямо, если нет, смотрит находится ли он на развилке, если вариантов много, он помечает на карте, что тут недообследованный путь и продолжает свое движение, если же он в тупике, строит кратчайший маршрут до последней (ближайшей) неисследованной развилки и продолжает его обследование уже с нее. Окончание мисси же может быть вызвано окончанием памяти, выходом из строя какого-либо компонента, или

же зацикливанием карты (в данном случае будет считаться, что робот полностью обследовал помещение, рисунок 14).

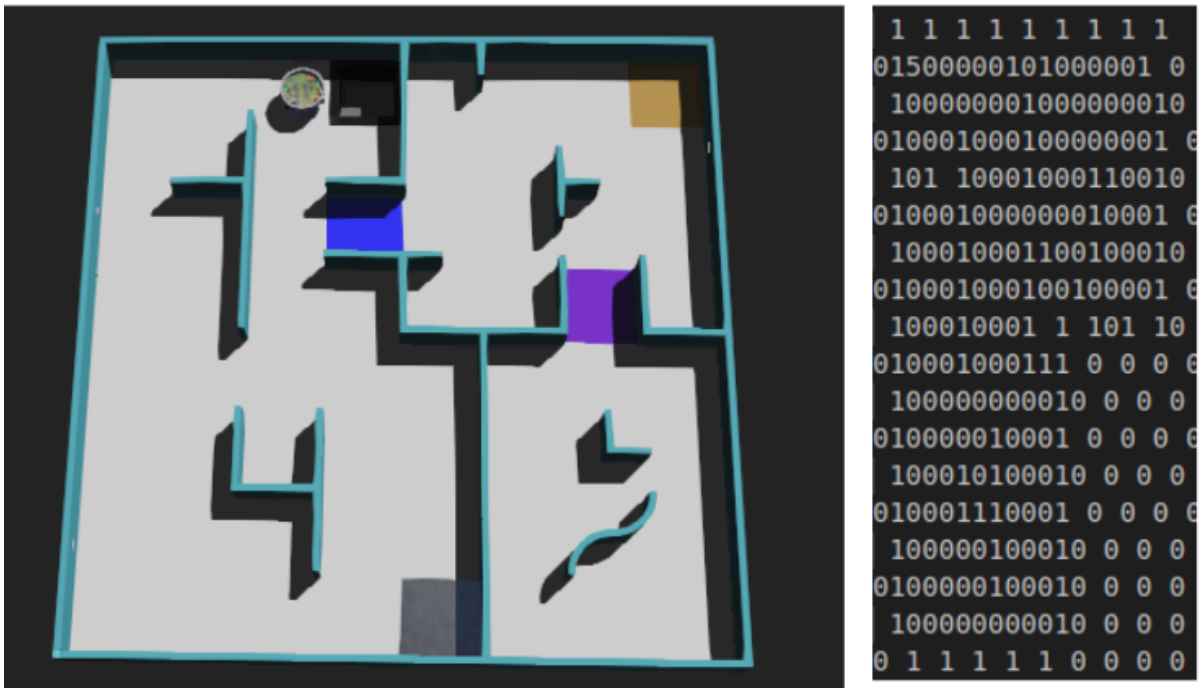


Рисунок 14 - результат картографирования робота

Для программирования RPI, также были составлены несколько разных программ, которые позже собирались в одну. Так, для более точного определения возможных знаков или надписей на языке C++ был разработан алгоритм чтения букв, который мы назвали «метод точек», а для быстрого реагирования на изменения в окружающей среде был обучен фреймворк искусственного интеллекта YOLO v8, написанный на языке Python.

Для определения видимых жертв нам понадобилось проводить бинаризацию на цвета, характерные для цветных и буквенных жертв, далее – само определение знаков/чтение букв. Однако мы не можем предсказать освещение во время миссии, по-этому нам понадобилось разработать алгоритм смены параметров автобинаризации в зависимости от освещения на поле. Алгоритм, который мы придумали ищет на изображении самый светлый, самый темный, среднестатистическое значение пикселя и фор- мулой рассчитывает данные для бинаризации на черный (чем темнее освещение на поле, тем больше делителей). Таким образом, робот может находить линию даже в

сумраке (рисунок 15).

Примерная формула, которую мы используем на псевдокоде:

текущий_пиксель<(min+ср_знач)/2+ min

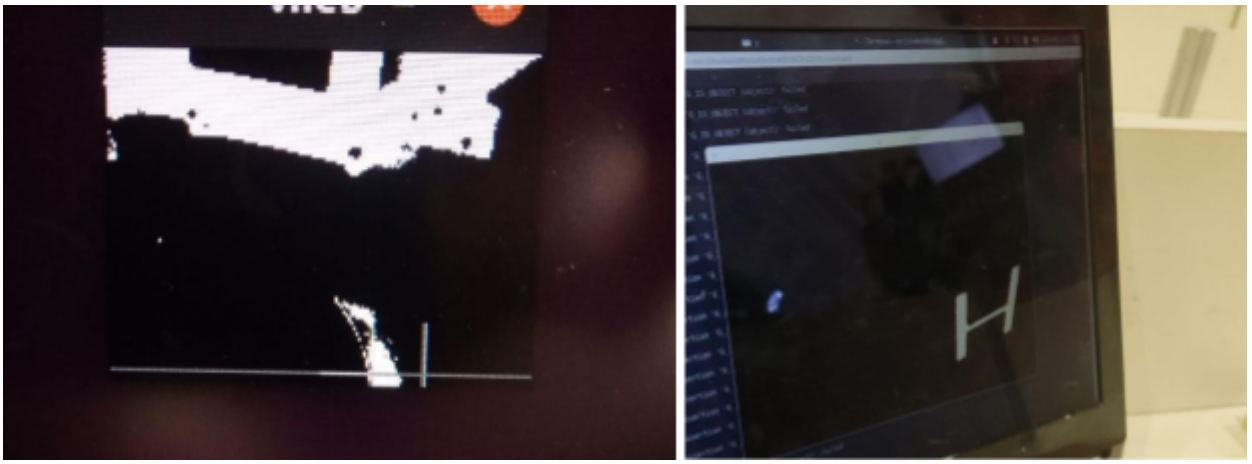


Рисунок 15 - результат работы алгоритма автобинаризации

Само обнаружение же происходит с помощью камеры и библиотеки OpenCV 4. Для начала настраивается бинаризация (смотрится по количеству цветных пикселей) для одного из цветов: красного, жёлтого и чёрного. После бинаризации определяем контуры и проверяем самые большие контуры, подходят ли они под параметры знаков. Если обнаружены чёрные контуры, подходящего размера, то вычисляем 3 точки прямоугольника этого контура: верхнюю срединную и нижнюю срединную, с помощью них мы и определяем какая буква и ищем в них контур на чёрной бинаризации: если верхний и нижний - черные, то это s, если нижний и верхний - не черные - то это H, а если нижний – чёрный - то это U (рисунок 16, 17). Как можно заметить, данный метод основывается на различие начертания букв. Сейчас нам достаточно всего 2-х точек, так как мы читаем всего 3 буквы, однако с увеличением количества букв, необходимых для чтения нам достаточно просто увеличить количество точек и их расположение.



Рисунок 16 – «метод точек» наглядно

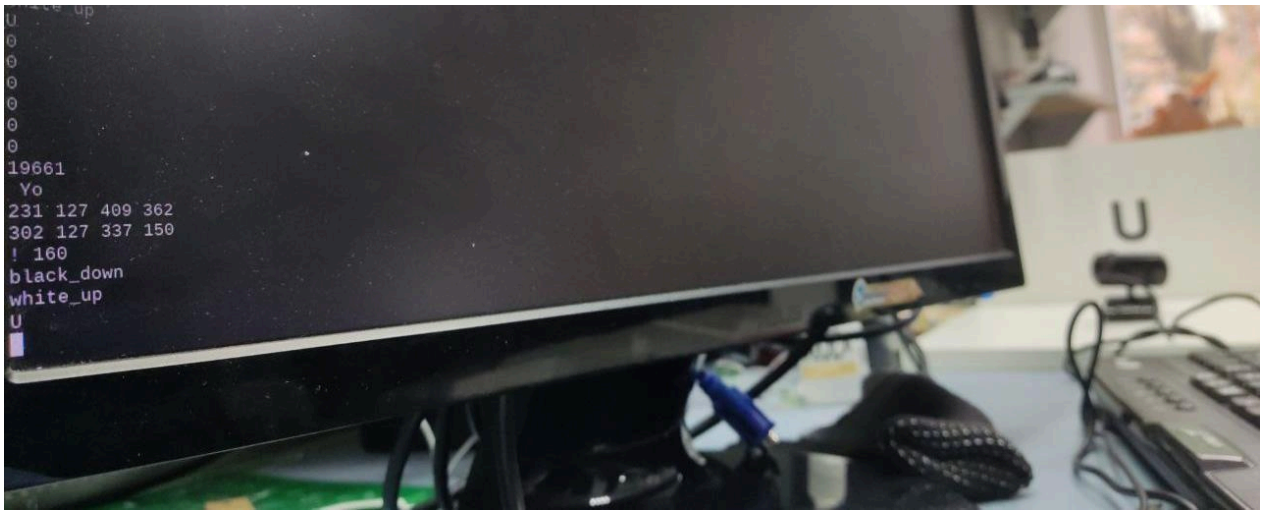


Рисунок 17 – результат работы алгоритма «метод точек»

Еще одним вариантом распознавания букв, а также вариантом распознавания жертв стало обучение фреймворка искусственного интеллекта YOLO v8 (рисунок 18). Механизм работы нашей программы предельно простой: у нас имеется заранее натренированная модель на датасете поля, преобразуем каждый кадр в тензор, сравниваем с тензора и в модели, получаем боксы детектирования, если условный "коэффициент сходства" в подобном боксе больше определённого значения, то узнаем, что за жертва/буква привязана к нему. Для реализации этого алгоритма собственно нужно было подготовить сам датасет: снять фото с камеры робота, и с помощью спец инструментов провести разметку. Так как мы использовали нейросеть YOLO v8, то и тренировали мы модель YOLO v8 (n/m/x) на нашем датасете. Далее экспортировали в расширение torchscript, передавали файл распберри, т.к. программируем RPI на C++, а YOLO обучается на Python - использовалась библиотека libtorch.



Рисунок 18 – итог работы алгоритма распознавания на YOLO v8

Тестирование ПО, вне симулятора производилось непосредственно на поле

при выполнении миссии. Для обучения YOLO мы запустили робота и сохраняли кадры с камеры на роботе раз в 0,5/1 секунд. А позже для тестирования распознавания жертв и букв мы использовали распечатанные буквы, так как это удобнее, чем на поле.

Тестирование робота

Тестирования прохождения самой миссии проводили на полигонах для соревнований макетов роботов спасателей. Полигоны собраны из ЛДСП и симулируют выполненную миссию (рисунки 19, 20, 21), также на полигонах можно встретить препятствия, мусор, подъемы-спуски, рампы и лестницы. Робот же выполнял миссии в нескольких режимах: с передачей информации людям, и обособленно, без вмешательств, так как нам не интересны показания датчиков, нам было достаточно подключить планшет к RPI и отслеживать, что она пишет в командную строку. Таким образом мы можем оперативно получать информацию о нахождении жертв прямо на планшет.

Если вдаваться в анализ работоспособности проекта, можно сделать вывод что он достаточно эффективен для выполнения задач поставленных регламентом, однако еще требует доработок (см. Заключенние).

Видеозаписи большей части испытаний можно посмотреть на нашем облачном диске (см. ссылки на авторов).



Рисунок 19, 20 – примеры полей для испытаний

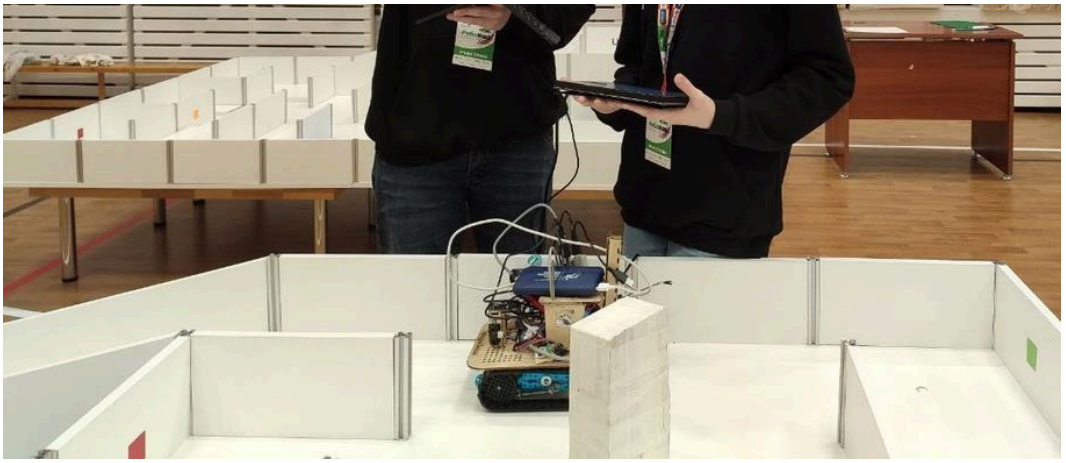


Рисунок 21 – отладка робота на поле

Таким образом, после доработки промежуточных результатов разработки на каждом из этапов, были пройдены несколько этапов разработки и итераций робота (рисунки 22-34), получила и протестировала рабочий эксперементальный прототип робота для помощи проведения поисково-спасательных операций (рисунок 38).

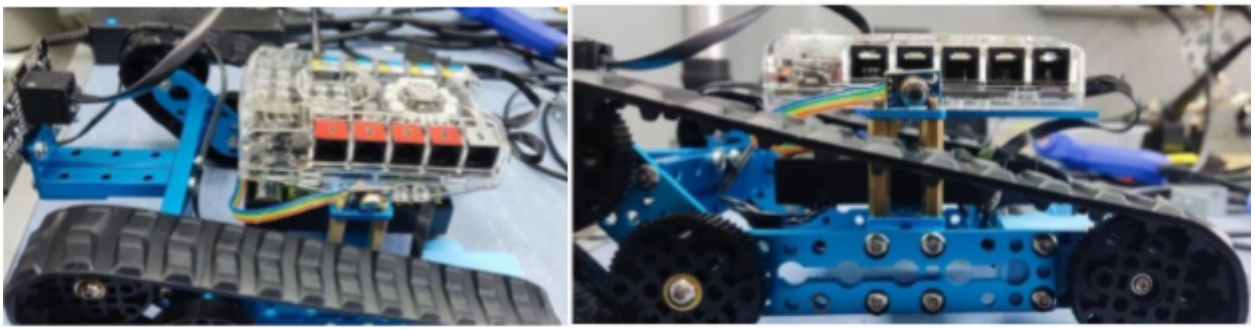


Рисунок 22 – первая версия робота

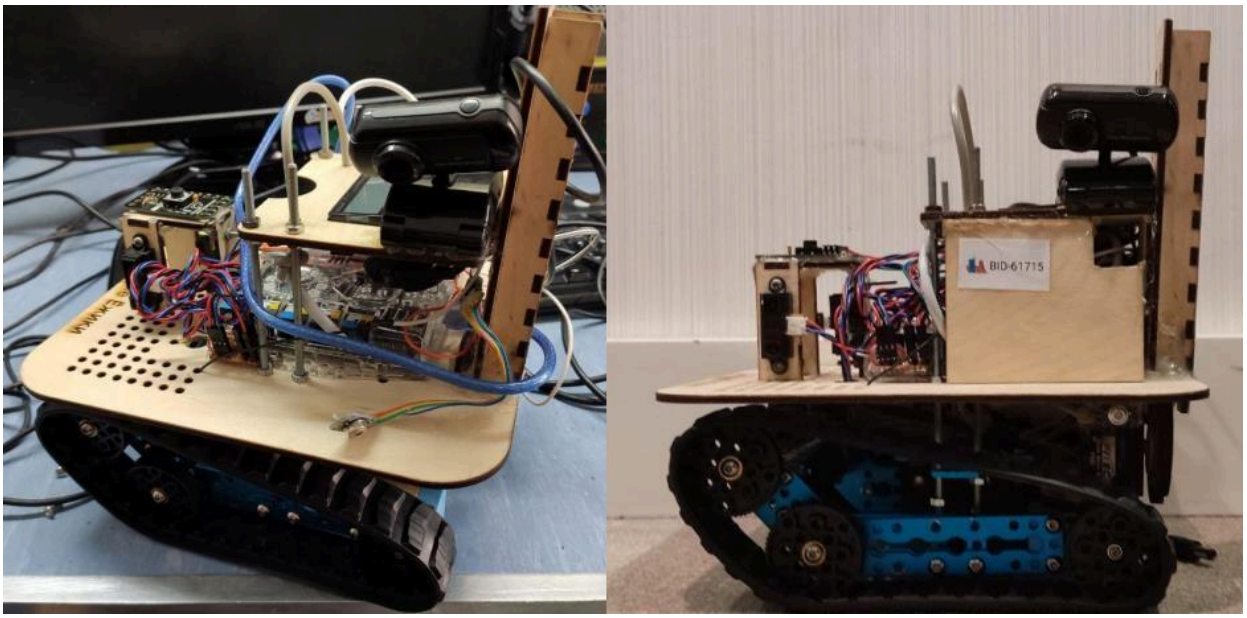


Рисунок 23 – вторая версия робота

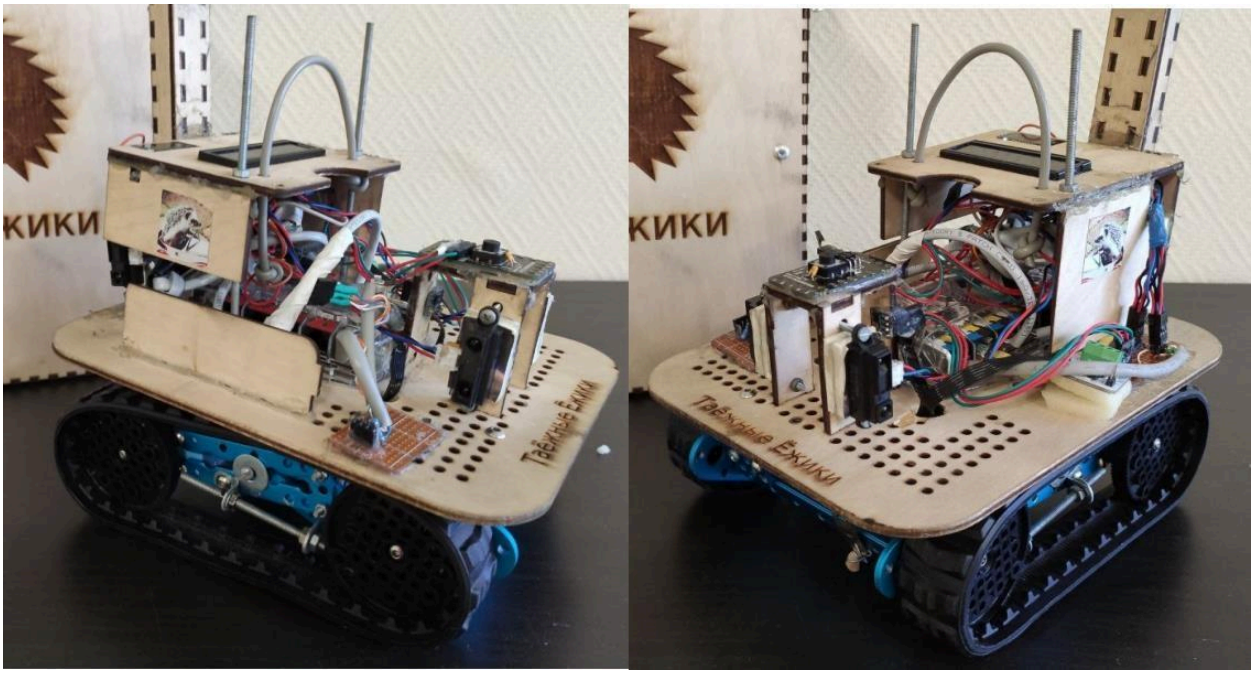


Рисунок 24 – третья версия робота

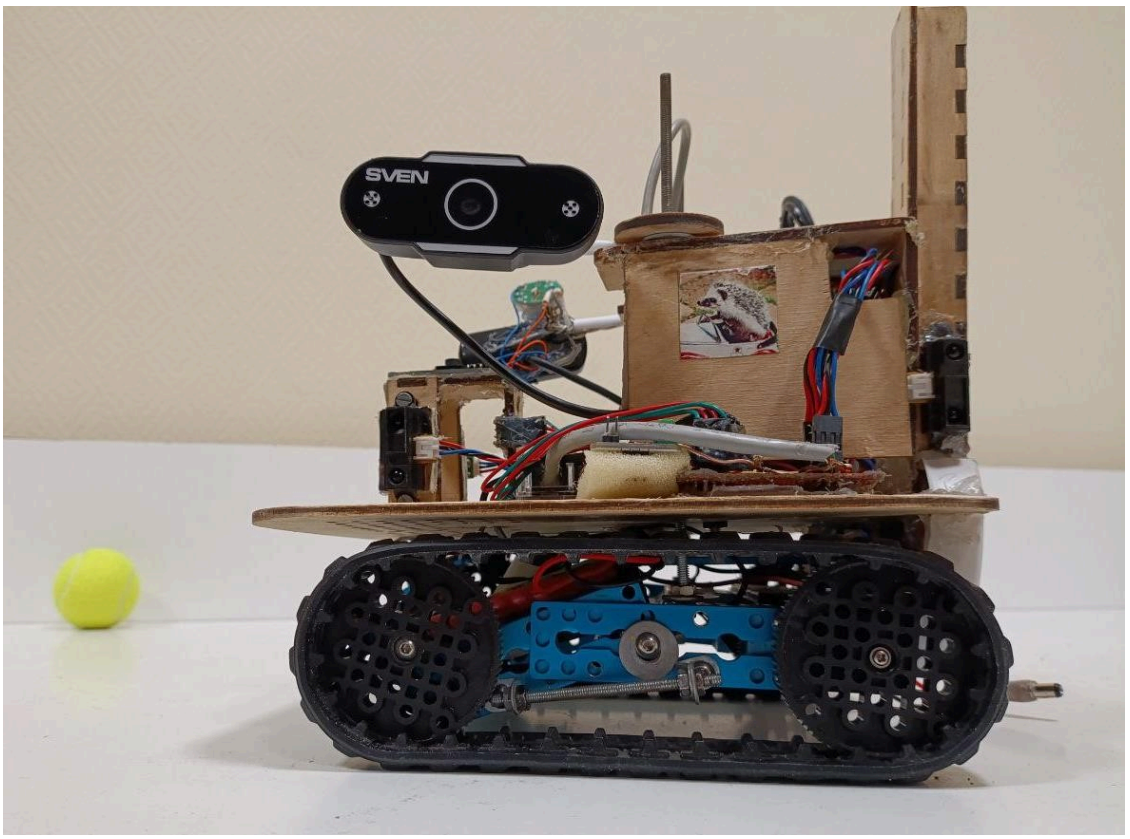


Рисунок 25 – четвертая версия робота

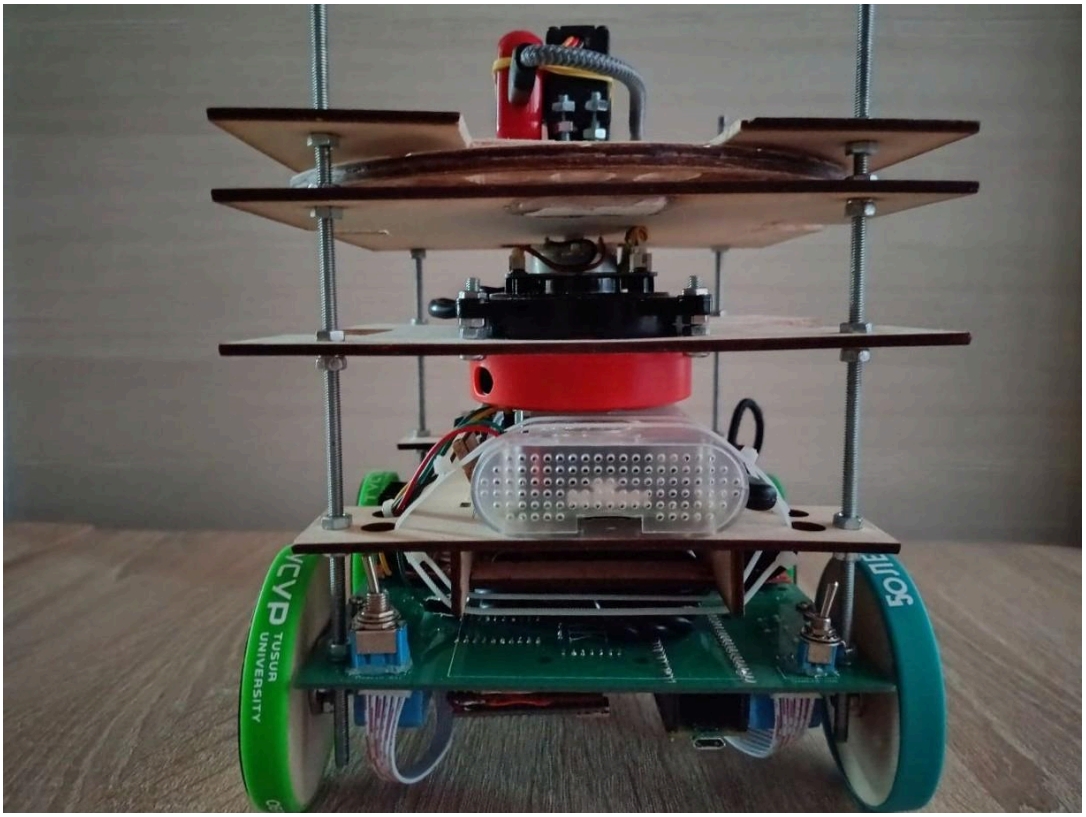


Рисунок 26 - пятая версия робота

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

В результате проделанной работы желаемый результат был достигнут, был разработан и протестирован рабочий прототип робота для проведения поисково-спасательных операций. Данный робот способен в автономном режиме обследовать территорию, строить его карту и определять наличие знаков опасности.

Проблемы, с которыми мы столкнулись на этапе реализации

Во время разработки и модифицирования нашего робота команда столкнулась со следующими проблемами.

1) При разработке чертежей робота, необходимо было сразу предусмотреть все необходимые отверстия и крепежи, из-за постоянной модификации конструкции, необходимо заново переделывать их. (основной платформы у нас 3 варианта, последнюю из которых также пришлось модифицировать).

2) Освещение на тестовых полях было очень плохим и к тому же менялось с течением суток, настройка бинаризации очень долгий процесс, из-за чего было принято решение разработать самодельный алгоритм автобинаризации.

3) Нам пришел не рабочий расширитель портов и в предверье соревнований нужно было решить проблему с портами, в результате как временный расширитель портов взяли Arduino Nano.

4) Изначально в качестве основного контроллера мы работали с платой meAuriga, но она не смогла потянуть алгоритмы картографирования, а также было большое желание перейти на полноприводного робота, в результате в качестве эксперимента мы заменили контроллер на Raspberry Pi PICO.

5) Так как лидар с которым мы работаем предназначен для робота пылесоса в сети нет открытых библиотек, протоколов и описаний работы этого лидара, с этой задачей нам помог Дементьев Юрий Николаевич, который предоставил архивы студенческий хакатонов с протоколами лидаров разных моделей.

6) Зачастую у Raspberry сбивается время, и программа не компилируется, выводя ошибку «Время изменения файла находится в будущем», для решения данной проблемы мы копируем новый каталог под новым временем и сносим старый с неправильным.

7) Для масштабной и подробной карты необходимо много памяти, которой сильно не хватало. Решением стало использование методов побитовых операций и сжатие переменных до 1 байтных.

8) Raspberry постоянно поедает sd карты, они ломаются и порой даже не поддаются форматированию, мы не смогли до конца решить данную проблему, поэтому просто делаем бэкапы и заменили microSD на жесткий диск

9) Обучение нейросетей, оказалось довольно удача затратным занятием, так как очень сложно предугадать, сколько ресурсов вам понадобится, чтобы нейросеть обучилась, но не переобучилась. В связи с чем было сделано множество вариаций этой нейросети и со временем научились это рассчитывать.

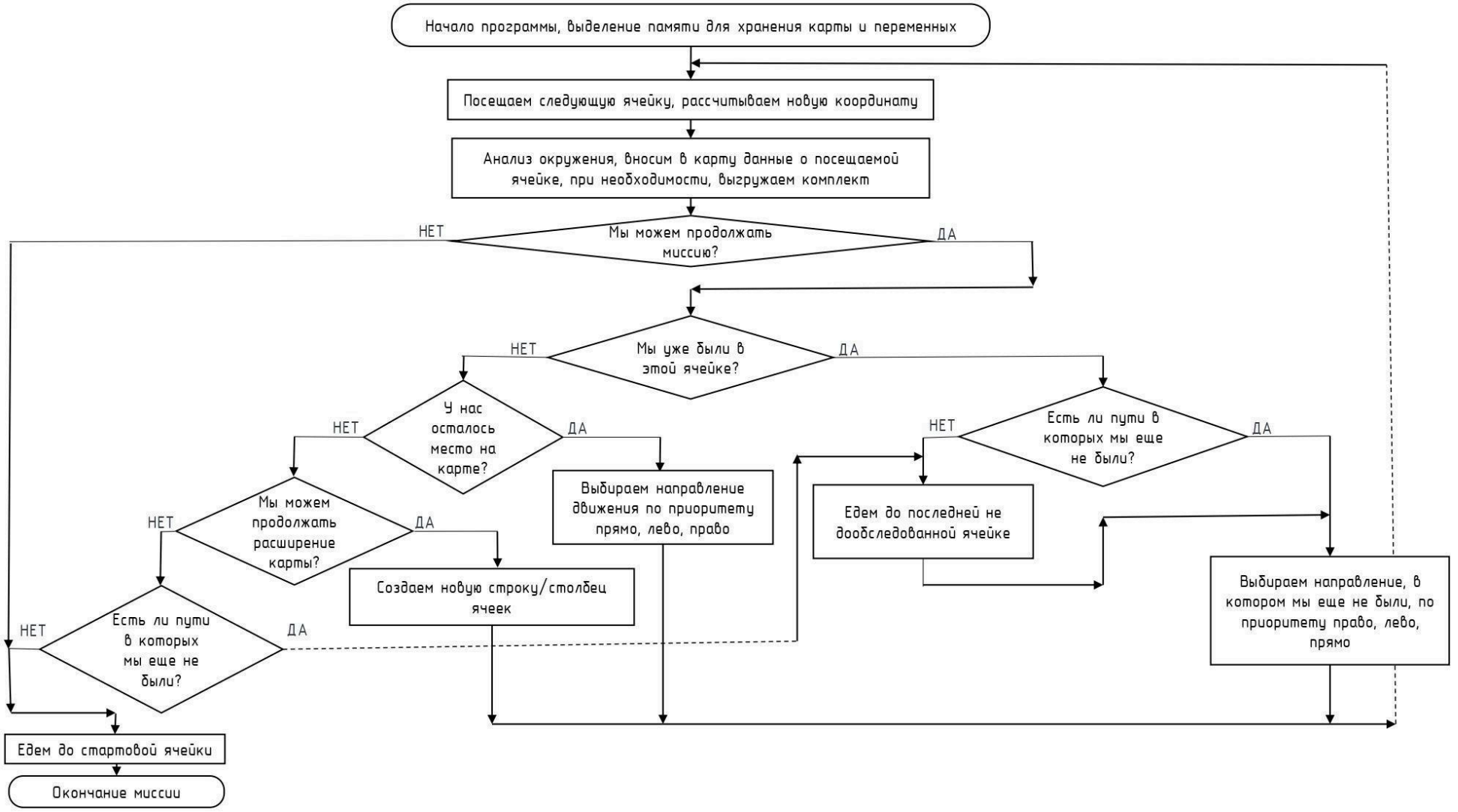
10) Технологии машинного обучения тратят довольно много ресурсов, к тому же мы не отказывались от метода точек, в связи с чем возникла проблема с использованием Raspberry Pi Zero и произошёл переход на Raspberry Pi 4.

Планы на дальнейшую доработку

Так как мы перешли с обычных ИК дальномеров на лидар мы хотим продолжать работу над этим проектом в сторону улучшения качества построенных нами карт, так как по сути теперь у нас не 5 дальномеров, а все 360. Кроме того, текущие алгоритмы картографирования и навигации работают довольно медленно, сейчас робот может долго стоять на перекрестках и рассчитывает маршрут, в качестве эксперимента мы уже перешли на RPI PICO, в дальнейшем, если тестирование покажет небольшую разницу с предыдущей итерацией робота, нужно будет сильно заняться оптимизацией. Также командой планируется и дальше изучать компьютерное зрение и нейросети.

Приложение 1

Блок-схема алгоритма навигации



Ссылки на нас

Здесь вы можете подробнее прочитать про этот и другие проекты команды «Таёжные Ёжики», изучить чертежи, код, датасеты, а также посмотреть на все испытания и тесты робота.

https://github.com/Tsygankova-Maria/TE2022/tree/main (наш GitHab)

https://drive.google.com/drive/folders/1fmRRXI1r2HDXeKdHgOfJ3aFNQoEvYZH1?usp=sharing (облачный диск, где собираются видео испытаний робота)

https://drive.google.com/drive/folders/1vHIvbDYvG99-oNjlotWKDVUvc7Kxek05?usp=sharing (облачный диск, где собирается техническая документация робота)

https://www.youtube.com/@user-cj5jf6zu6l (наш YouTube)

https://www.youtube.com/@SergKosachenko (YouTube нашего тренера, здесь можно посмотреть некоторые записи испытаний)

Мы каждый год публекуем свои материалы и будем рады помочь Вам при подготовке!

Контактные данные авторов

М.С. Цыганкова: maria.tsygankova.tftl@gmail.com

Г.А. Пильщиков: p.g.a.2019@mail.ru

С.В. Косаченко: kosachenkosv@yandex.ru